ECE131L Group 5 Project Report

This project is a recreation of the game Wordle by New York Times that was developed in the programming language C and is played inside of a terminal. The objective of this game is to correctly guess a randomly selected 5 letter word in 6 guesses or less. As players play this game, they are given clues to the mystery word from each guess that they make by having the letters be highlighted as “almost” or “correct” based on the letters position in the word. This program is ultimately designed for people who want a fun game to play in the terminal, and it has extra features such as adding words that your choice to the word library, and game history logging and reporting.

For the design of this program, we are using multiple different functions and structures to keep our game logic and data sets as organized and modular as possible. Our program includes two structures, one of them is the MyGame structure, which stores data about the game state such as the mystery word and the number of turns taken. The other structure is named GameHistory, and it stores statistics from previous games such as the number of guesses, the mystery word, and if the player won or lost. We also have implemented tons of Loops and Arrays since this game is entirely based on strings, and some examples where you’ll find arrays are for storing words and guesses, letter matching, and managing file I/O.

We have done extensive testing on this program, and it all started with our test plan that we developed. It is very important that we test all system functions through pushing them to their limits and making sure that we can patch any bugs that our program may have. However, we put special attention into testing and pushing the limits of our play function, since it is the highlight of our program. Some of the test cases included looking at valid and invalid user inputs, testing boundaries like the letter count and guess count, testing case sensitivity for user inputs, testing user inputs that already exist in the library, and viewing game history when there haven't been any games played. All these measures were taken to make sure that the program would always have appropriate outputs, and so any user would be able to play this game without any game troubles.

One of our biggest challenges in this project was handling cases where repeated letters were involved in the guessing word, and the actual word. Initially the program was incorrectly marking words with multiple letters since it would count all instances of the letter from the guess word that was in the mystery word as "almost" besides the letter in the correct position which is untrue, and deceptive to players. We ended up resolving this issue by implementing a letter-count loop in our play function so that each letter from the guess word was matched to the letters of the mystery word only once.